

Assignment1 - report

2022036208 인범진

Design

- 새로운 system call(ps.c) 구현을 위해서 우선 system call이 호출되고 실행되는 과정에 대해 알아보았다.
- `ps` 라는 이름의 system call 과 이를 호출할 수 있는 같은 이름의 user program을 만들었다.
 - ps system call 구현
 - 우선 새로운 `ps` 라는 함수를 kernel/proc.c에 구현함
 - kernel/defs.h에 다른 c파일 처럼 visible하게 만듦
 - kernel/sysproc.c에 ps system call을 위한 wrapper function을 만듦
 - kernel/syscall.h에 wrapper function number를 등록함
 - 이를 통해 kernel/syscall.c에서 wrapper function을 참조함
 - ps user program 구현
 - ps.c를 user 파일에 추가
 - user/user.h에 user program을 선언함
 - user/usys.pl 코드를 추가하여, `ps` 호출을 통해 `ps` 시스템 콜 번호를 레지스터에 넣고, `ecall`을 실행함
 - 정리하면 user파일에 ps.c를 새로 추가하고, kernel/proc.c, defs.h, sysproc.c, syscall.h와 user/user.h, usys.pl, Makefile 파일을 수정하였다.

Implementation

- ps system call 구현

- proc.c 파일의 `procdump` system call을 참고하여, `ps` system call을 구현하였다.

```
665 void
666 procdump(void)
667 {
668     static char *states[] = {
669         [UNUSED]    "unused",
670         [USED]       "used",
671         [SLEEPING]   "sleep ",
672         [RUNNABLE]   "runble",
673         [RUNNING]    "run  ",
674         [ZOMBIE]     "zombie"
675     };
676     struct proc *p;
677     char *state;
678
679     printf("\n");
680     for(p = proc; p < &proc[NPROC]; p++){
681         if(p->state == UNUSED)
682             continue;
683         if(p->state >= 0 && p->state < NELEM(states) && states[p->state])
684             state = states[p->state];
685         else
686             state = "???";
687         printf("%d %s %s", p->pid, state, p->name);
688         printf("\n");
689     }
690 }
```

procdump(original code)

- `procdump`에서는 프로세스 테이블을 순회하여, 커널이 관리하는 모든 프로세스를 확인한다

- `p = proc` 을 이용해 프로세스 테이블을 p에 할당하여, 각 프로세스에 참조하는 것을 가능하도록 한다
- `ps` 또한 for 문을 이용해 xv6에서 동시에 존재할 수 있는 최대 프로세스 개수(== `NPROC`)만큼 테이블을 순회한다
- 단, argument가 있는 경우와 없는 경우, 두가지 경우에 대비하여 각각 반복문을 다르게 작성하였다
- 또한, active process, 즉, (`procdump` 함수 기준) 프로세스가 `unused` 가 아닌 경우에만 이를 출력하도록 한다
 - argument가 존재하나, target process가 `unused` 인 경우에도 이를 출력하지 않는다
- 이에 따라 `ps` 함수는 다음과 같이 구현했다

```
int
ps(int pid)
{
    static char *states[] = {
        [UNUSED]  "unused",
        [USED]     "used",
        [SLEEPING] "sleep",
        [RUNNABLE] "runble",
        [RUNNING]  "run",
        [ZOMBIE]   "zombie"
    };
    struct proc *p;

    if(pid == 0){
        for(p = proc; p < &proc[NPROC]; p++){
            if(p->state != UNUSED)
                printf("%d,%s,%s\n", p->pid, states[p->state], p->name);
        }
        return 0;
    } else{
        for(p = proc; p < &proc[NPROC]; p++){
            if(p->pid == pid && p->state != UNUSED){
                printf("%d,%s,%s\n", p->pid, states[p->state], p->name);
                return 0;
            }
        }
        return 1; //target process 없는 경우
    }
}
```

ps(added code)

- `states` 는 `procdump` 의 `states` 를 참고했다
- pid 값이 0인 경우, argument가 없는 경우를 다룬다
- 과제 명세에 따라 target process가 없는 경우, 함수 return 값을 1, 있는 경우에 return 값을 0으로 하였다.

```
104 | int ps(int);
```

kernel/defs.h 에 방금 만든 ps 함수 추가

- 이 언급을 통해 kernel 파일 안 다른 c 파일(sysproc.c)도 이 함수에 대해 접근할 수 있도록 한다

```
111 | uint64
112 | sys_ps(void)
113 | {
114 |     int pid;
115 |
116 |     argint(0, &pid);
117 |
118 |     return ps(pid);
119 | }
```

sysproc.c 속 sys_ps 구현

- wrapper function인 (`sys_ps`)를 만들어서 kernel 내부 `ps` 를 호출하도록 한다
- `argint` 함수를 이용하여, pid 값을 인자의 값으로 세팅한다 (→ `argint` 의 기능은 `sysfile.c`를 참고함)
 - kernel에서 0번째 인자(a0)를 pid 값으로 가져옴
- 리턴 값에 `ps` 함수를 호출하여 해당 pid에 맞는 값을 호출한다

```

17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
23 #define SYS_ps 22

```

syscall.h에 방금 만든 SYS_ps를 등록

```

132
133 void
134 syscall(void)
135 {
136     int num;
137     struct proc *p = myproc();
138
139     num = p->trapframe->a7;
140     if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
141         // Use num to lookup the system call function for num, call it,
142         // and store its return value in p->trapframe->a0
143         p->trapframe->a0 = syscalls[num]();
144     } else {
145         printf("%d %s: unknown sys call %d\n",
146             p->pid, p->name, num);
147         p->trapframe->a0 = -1;
148     }
149 }

```

syscall.c 에서 방금 헤더의 번호를 이용하여, 해당 system call을 참조

- 이 등록을 통해 syscall.c 에서 `syscall` 의 배열에 `sys_ps` 에 해당하는 번호를 넣어서, 해당 system call 을 수행할수 있도록 한다

- ps user program

```

user > C ps.c > main
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  int main(int argc, char **argv){
6
7      printf("pid,state,name\n");
8      if(argc == 1)
9          ps(0);
10     else
11         for(int i = 1; i < argc; i++)
12             ps(atoi(argv[i]));
13
14     exit(0);
15 }

```

- ps.c 라는 user program을 생성
- argument가 없는 경우와 있는 경우로 나눠서 처리하였다

```

27 | int ps(int);

```

user.h에 ps 함수

- user.h 파일에 `int ps` 함수를 추가한다.
 - 이는 user level에서 system call 을 호출하기 위한 선언

```

45 | entry("ps");

```

usys.pl에 "ps"entry를 추가

- usys.pl에 "ps" entry를 추가한다

- usys.pl은 실행되면 usys.S 어셈블리 코드를 자동 생성한다
 - user program이 ps를 호출하면, user.h에 해당 함수의 선언을 확인 후
 - usys.S를 실행함
 - usys.S에서의 ps 어셈블리 코드를 실행함
- usys.S 파일이 실행되면 kernel/syscall.c의 `syscall` 함수가 실행된다
 - syscall.c의 syscall 함수에서 `num = p->trapframe->a7;` 이 코드는
 - usys.S에서 '`li a7, SYS_ps`'로 저장했던 system call 번호를 가져옴
- "usys.S가 ps의 시스템 콜 번호를 a7 레지스터에 넣고 ecall을 실행하면, syscall() 함수가 그 번호를 읽어서 해당 system call 함수를 실행한다"
- 추가로 Makefile의 `USPROGS` 항목에 `_ps\`를 추가하여 user/ps.c가 컴파일되어 xv6 내부에서 실행 가능하도록 한다

Results

```

inbeomjin@inbeomjin-ui-MacBookAir ~/assignment1-Ben9960 % make qemu
init: starting sh
$ ps
pid,state,name
1,sleep,init
2,sleep,sh
3,run,ps
$ ps 2
pid,state,name
2,sleep,sh
$ ps 1 2 4
pid,state,name
1,sleep,init
2,sleep,sh
$ ps 1 2 5
pid,state,name
1,sleep,init
2,sleep,sh
$ ps 1 2 7
pid,state,name
1,sleep,init
2,sleep,sh
7,run,ps
$ ps 10
pid,state,name
$

```

- \$ ps
 - 이 case를 이용해 argument 없이 실행된 경우, 모든 프로세스를 제대로 표시하는지 확인한다
 - 1, 2, 3 까지의 pid를 가진 프로세스를 모두 정상적으로 출력하는 것을 볼 수 있다
- \$ ps 2
 - 이 case를 이용해 하나의 argument에 대해 그 target process를 제대로 출력하는지 확인한다
 - 2라는 pid를 가지는 프로세스를 정상적으로 출력한다
- \$ ps 1 2 4
 - 이 case를 통해 복수의 arguments들의 target process들을 제대로 출력하는지 확인한다
 - 1 2의 target process는 제대로 출력하였고, 4의 pid를 가지는 프로세스는 없으므로 출력하지 않는 것을 확인했다
- \$ ps 1 2 7
 - 이 case를 통해 복수의 arguments들의 target process들을 제대로 출력하는지 다시 한번 확인한다
 - 1 2 7 각각의 target process를 제대로 출력하는 것을 확인했다.
- 위 Case들을 통해 다른 많은 testCase에 대해 cover 할 수 있는 user program과 system call을 만든 것을 알 수 있다

Troubleshooting

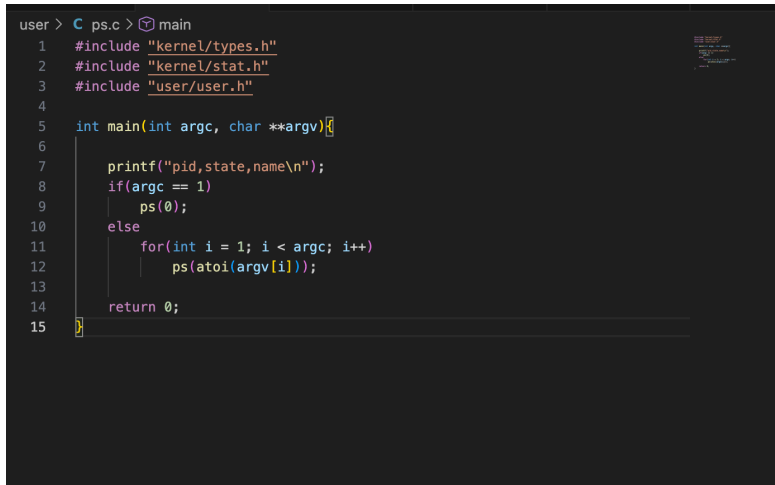
- sys_ps 함수와 ps 함수 그리고 user program인 ps에 대해 각각이 어떤 역할을 부여해야 하는지 헷갈리는 부분이 존재했다
- 그러나 기존에 구현했던 getppid 그리고 lab 내용을 통해 각각의 역할과 system call execution에서 어떤 순서로 실행되는지 파악하니, 보다 수월하게 코드를 짤 수 있었다
- 또한, argint 함수의 사용에 미숙하여, sys_ps 함수 코드를 짤 때, 어떻게 인자 값을 받아올지 고민했다

System Call Execution Flow

1. user program 시작

```
$ ps 1 2 5
```

- ps.c user program이 실행됨



```
user > C ps.c > main
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  int main(int argc, char **argv){
6
7      printf("pid,state,name\n");
8      if(argc == 1)
9          ps(0);
10     else
11         for(int i = 1; i < argc; i++)
12             ps(atoi(argv[i]));
13
14     return 0;
15 }
```

- argument 개수가 복수인 경우이므로 else 문 실행
- ps(1), ps(2), ps(5) 차례로 호출
 - 현재 init, sh(shell), ps 프로세스가 실행 중

2. system call refer

- user/usys.S 파일이 usys.pl 파일을 통해 자동 생성됨

```
sub entry {
    my $prefix = "sys_";
    my $name = shift;
    if ($name eq "sbrk") {
        print ".global $prefix$name\n";
        print "$prefix$name:\n";
    } else {
        print ".global $name\n";
        print "$name:\n";
    }
    print " li a7, SYS_{$name}\n";
    print " ecall\n";
    print " ret\n";
}
```

```
entry("ps");
```

- a7 register에 시스템 콜 번호를 넣음
- 이후 커널로 진입
- ecall 명령어 실행(이를 통해 User mode 에서 Kernel mode로 진입)
- 이후 trap.c 파일 실행 후 syscall() 실행
 - trap.c파일이 실행되면 trap의 원인을 분석함

3. syscall() 실행

```
void
syscall(void)
{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->a7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        // Use num to lookup the system call function for num, call it,
        // and store its return value in p->trapframe->a0
        p->trapframe->a0 = syscalls[num]();
    } else {
        printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}
```

- `p->trapframe->a0 = syscalls[num]();`
 - 이 줄을 통해 num 번째 system call 수행
 - ps라는 system call이 실행되기 위해, syscall 배열에서 ps의 번호에 해당하는 원소를 가져와서 `p->trapframe->a0` 에 넣는다

```
// Prototypes for the functions that handle system calls.
extern uint64 sys_fork(void);
extern uint64 sys_exit(void);
.....
extern uint64 sys_ps(void);

// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
    [SYS_fork]    sys_fork,
    [SYS_exit]    sys_exit,
    .....
    [SYS_ps]      sys_ps,
};
```

- 같은 syscall.c 파일에 syscall 배열 가장 아래 `[SYS_ps] sys_ps` 값을 가져옴

4. sysproc.c 파일의 sys_ps 실행

```
uint64
sys_ps(void)
{
    int pid;

    argint(0, &pid);
```

```

    return ps(pid);
}

```

- sysproc.c의 새로 작성한 `sys_ps` 실행됨
- `argint` 함수를 통해 argument 값을 받아와서 pid값에 저장

5. proc.c 파일의 ps 함수 실행

```

int
ps(int pid)
{
    static char *states[] = {
        [UNUSED]    "unused",
        [USED]       "used",
        [SLEEPING]   "sleep",
        [RUNNABLE]   "runble",
        [RUNNING]    "run",
        [ZOMBIE]     "zombie"
    };

    struct proc *p;

    if(pid == 0){
        for(p = proc; p < &proc[NPROC]; p++){
            if(p->state != UNUSED)
                printf("%d,%s,%s\n", p->pid, states[p->state], p->name);
        }
        return 0;
    } else{
        for(p = proc; p < &proc[NPROC]; p++){
            if(p->pid == pid && p->state != UNUSED){
                printf("%d,%s,%s\n", p->pid, states[p->state], p->name);
                return 0;
            }
        }
        return 1; //target process 없는 경우
    }
}

```

- 변수 p가 `proc[]` 배열 순회
- pid 일치하는 프로세스 찾아 정보 출력
 - 여기서는 1, 2, 5에 해당하는 process 찾아 정보 출력
- 반환값: 0 (성공) 또는 -1 (실패)
 - 1, 2에 해당하는 process 있으므로 0을 반환
 - 5에 해당하는 process 없으므로 1을 반환

6. 유저 모드로 복귀

- trampoline.S
 - 커널에서의 작업을 끝내고 유저 모드로 돌아가기 위한 마지막 assembly 단계